



Contenido

1. Introducción	1
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Marco teórico	2
4. Metodología	2
5. Implementación en Python	2
5.1. Estructura del Proyecto	2
5.2. Conformacion del portafolio	3
5.3. Entrenamiento de la red neuronal	3
5.4. Cálculo del VaR con el modelo TimeGAN	3
5.5. Cálculo del VaR con metodos convencionales	3
6. Resultados	3
7. Conclusiones	3
8. Bibliografía	3

1. Introducción

El Valor en Riesgo (VaR) se ha establecido como una métrica fundamental en la gestión de riesgos financieros, permitiendo cuantificar la máxima pérdida esperada para un portafolio en un horizonte temporal específico y con un nivel de confianza determinado. Los métodos tradicionales para su cálculo incluyen el enfoque paramétrico, la simulación histórica y la simulación de Monte Carlo, cada uno con sus propias limitaciones, especialmente evidentes durante períodos de alta volatilidad.

Las Redes Neuronales Recurrentes (RNN) han demostrado una notable capacidad para modelar series temporales financieras gracias a su arquitectura que incorpora conexiones cíclicas, permitiéndoles capturar dependencias temporales complejas en datos secuenciales. Paralelamente, las Redes Generativas Adversarias (GAN) han revolucionado el aprendizaje automático con su enfoque competitivo entre generador y discriminador, mostrando un potencial significativo para la generación de datos sintéticos realistas.

TimeGAN combina ambas tecnologías, integrando un componente supervisado que obliga al generador a preservar explícitamente la dinámica temporal de los datos originales. Esta innovación resulta particularmente relevante para aplicaciones financieras, donde capturar correctamente las estructuras de dependencia temporal es crucial.

Este proyecto implementa TimeGAN para generar series temporales sintéticas de retornos financieros y las utiliza para calcular el VaR mediante simulación de Monte Carlo. Nuestra metodología comprende el preprocesamiento de datos históricos, el entrenamiento del modelo TimeGAN, la generación de trayectorias sintéticas y la comparación de las

estimaciones de VaR con métodos convencionales, buscando determinar si este enfoque proporciona mediciones de riesgo más robustas, en comparación con los métodos tradicionales.

2. Objetivos

2.1. Objetivo general

Implementar y evaluar la arquitectura TimeGAN como metodología alternativa para el cálculo del Valor en Riesgo (VaR) de un portafolio diversificado compuesto por acciones, bonos gubernamentales, ETFs y oro, determinando su eficacia frente a métodos convencionales de estimación.

2.2. Objetivos específicos

1. Diseñar una arquitectura de proyecto modular y reproducible en GitHub, siguiendo principios de código limpio y documentación rigurosa para facilitar su replicabilidad.
2. Configurar un entorno de desarrollo basado en conda con integración de Cursor AI, Python y TensorFlow 2.0 con Keras que garantice la reproducibilidad computacional del proyecto.
3. Adaptar e implementar la arquitectura TimeGAN tomando como base el repositorio original desarrollado por gusxo (<https://github.com/gusxo/TimeGAN-keras>), que ofrece una implementación eficiente en Keras de la arquitectura propuesta por Yoon et al. (2019). Esta implementación será integrada dentro de un flujo de trabajo completo para el cálculo dinámico del VaR, permitiendo la modificación paramétrica del portafolio analizado según las necesidades del usuario.
4. Calcular el VaR del mismo portafolio utilizando metodologías convencionales (paramétrica, simulación histórica y Monte Carlo tradicional) como base comparativa.
5. Evaluar comparativamente la precisión y robustez de las diferentes metodologías mediante métricas cuantitativas adecuadas, con especial énfasis en su comportamiento ante escenarios de alta volatilidad y eventos extremos.

3. Marco teórico

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue. Ut in risus volutpat libero pharetra tempor. Cras vestibulum bibendum augue. Praesent egestas leo in pede. Praesent blandit odio eu enim. Pellentesque sed dui ut augue blandit sodales. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aliquam nibh. Mauris ac mauris sed pede pellentesque fermentum. Maecenas adipiscing ante non diam sodales hendrerit.

4. Metodología

A

5. Implementación en Python

5.1. Estructura del Proyecto

La estructura del repositorio se organiza de la siguiente manera:

```
TimeGAN-VaR/  
├── data/  
│   └── input/                # Datos originales para conformar el portafolio
```

```

├── processed/           # Retornos logarítmicos procesados del portafolio
├── output/             # Resultados y métricas exportadas del análisis
├── images/             # Visualizaciones y diagramas generados
├── models/             # Modelos TimeGAN entrenados (.h5)
├── notebooks/         # Cuadernos Jupyter interactivos
├──   ├── implementacion.ipynb # Notebook principal
├──   └── yfinance.ipynb      # Exploración de datos de Yahoo Finance
├── src/               # Código fuente modular del proyecto
├──   ├── data/
├──   │   ├── __init__.py
├──   │   ├── loader.py      # Carga y validación de datos financieros
├──   │   ├── transform.py   # Transformaciones para series temporales
├──   │   └── windowing.py   # Creación de ventanas para entrenamiento
├──   ├── models/
├──   │   ├── __init__.py
├──   │   ├── timegan.py     # Arquitectura TimeGAN completa
├──   │   └── var_model.py   # Cálculo de VaR con distintas metodologías
├──   ├── utils/
├──   │   ├── __init__.py
├──   │   ├── evaluation.py  # Métricas de evaluación y validación
├──   │   └── visualization.py # Generación de gráficos y visualizaciones
├──   └── __init__.py       # Inicialización del paquete principal
├── tex/
├──   └── proyecto_final.tex
├── .gitignore          # Archivos y directorios excluidos del repositorio
├── environment.yml     # Especificación del entorno Conda
├── LICENCE             # Licencia MIT del proyecto
└── README.md           # Documentación principal

```

5.2. Conformacion del portafolio

5.3. Entrenamiento de la red neuronal

5.4. Cálculo del VaR con el modelo TimeGAN

5.5. Cálculo del VaR con metodos convencionales

6. Resultados

A

7. Conclusiones

a

8. Bibliografía

- [1] J. Yoon, D. Jarrett y M. van der Schaar, "Time-series Generative Adversarial Networks," en *Advances in Neural Information Processing Systems*, 2019, págs. 5508-5518.
- [2] J. Yoon, *TimeGAN: Time-series Generative Adversarial Networks*, <https://github.com/jsyoon0823/TimeGAN>, 2019.
- [3] gusxo, *TimeGAN-keras: Implementation of TimeGAN with Keras*, <https://github.com/gusxo/TimeGAN-keras>, 2021.